

Bash – From zero to working knowledge

Day 4

Alessandro Sciarra

Z02 – Software Development Center

25 June 2026

Topics of the day

- 1 Asynchronous commands
- 2 Traps
- 3 Error handling
- 4 Sed (and Awk)

Exploring the dark side of Bash

Today we will finalise your knowledge adding a few powerful bricks that will provide you with lots of versatility and great abilities!

Asynchronous commands



Huge magma bombs

Processes in the terminal

 Bash manual v5.2 section 7

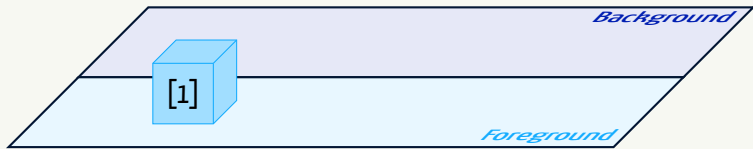
- The `ps` command displays information about (a selection of) the active processes
- A **process** is «an instance of a computer program that is being executed» – Wikipedia
- The shell associates a **job** with each pipeline
- It keeps a table of currently executing jobs, which may be listed with the `jobs` builtin
- When Bash (in interactive mode*) starts a job asynchronously (i.e. in background), it prints a line that looks like

```
$ sleep 30 & # <- The ampersand symbol starts  
[1] 25647    # a process in background
```

indicating that this job is job number 1 and that the process ID of the last process in the pipeline associated with this job is 25647 { We will come back to the process ID later }

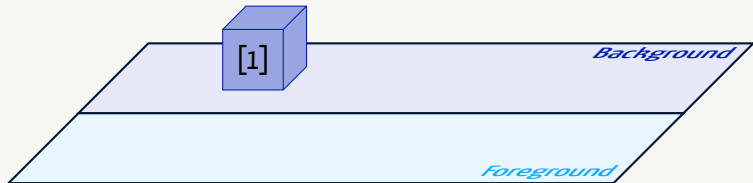
- All of the processes in a single pipeline are members of the same job
- Bash uses the job abstraction as the basis for job control

Job control: A graphical overview



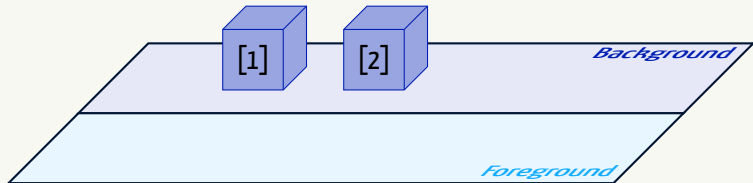
```
1 $ jobs
2 $ emacs Loops.tex    # <- each pipeline is a fg job by default
3 # Shell is blocked till you have the emacs process
4 # in foreground. You can close it to get the shell back.
```

Job control: A graphical overview



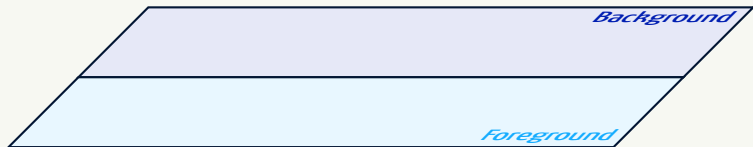
```
1 $ jobs
2 $ emacs Loops.tex    # <- each pipeline is a fg job by default
3 # Shell is blocked till you have the emacs process
4 # in foreground. You can close it to get the shell back.
5 $ emacs Loops.tex &  # <- use the ampersand to start it in bg
6 [1] 5642
7 $ jobs
8 [1]+  Running                  emacs Loops.tex &
```

Job control: A graphical overview



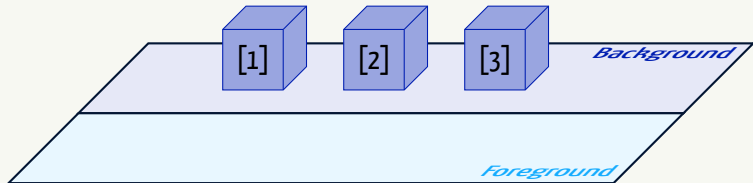
```
1 $ jobs
2 $ emacs Loops.tex      # <- each pipeline is a fg job by default
3 # Shell is blocked till you have the emacs process
4 # in foreground. You can close it to get the shell back.
5 $ emacs Loops.tex &    # <- use the ampersand to start it in bg
6 [1] 5642
7 $ jobs
8 [1]+  Running                  emacs Loops.tex &
9 $ sleep 30 &
10 [2] 20943
11 $ jobs
12 [1]-  Running                  emacs Loops.tex &
13 [2]+  Running                  sleep 30 &
```

Job control: A graphical overview



```
14 # After half a minute and closing emacs
15 [2]+  Done                sleep 30
16 [1]+  Done                emacs Loops.tex
```

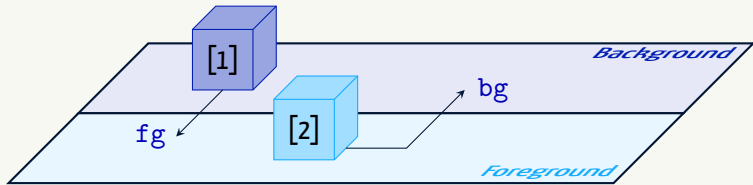
Job control: A graphical overview



```
14 # After half a minute and closing emacs
15 [2]+  Done                      sleep 30
16 [1]+  Done                      emacs Loops.tex
17 # You can start many jobs in background
18 $ for index in 30 60 90; do sleep ${index} & done && unset index
19 [1] 23559
20 [2] 23560
21 [3] 23561
22 $ jobs
23 [1]  Running                    sleep ${index} &
24 [2]  Running                    sleep ${index} &
25 [3]+ Running                    sleep ${index} &
```

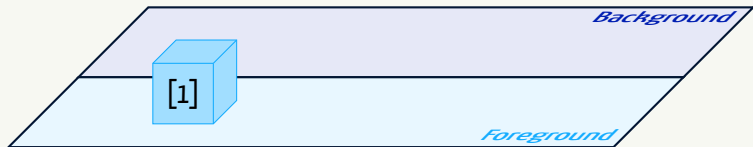
No semicolon!

Job control: A graphical overview



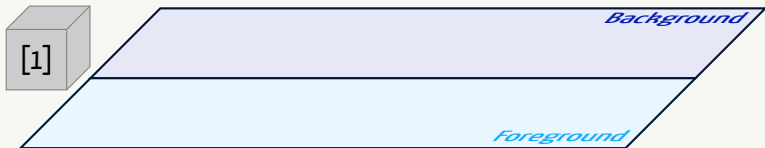
- Every running process can receive signals during its execution
- Some signals are bound to common keyboard shortcuts
 - CTRL-Z** sends SIGTSTP to the foreground job { usually suspending it }
 - CTRL-C** sends SIGINT to the foreground job { usually terminating it }
 - CTRL-** sends SIGQUIT to the foreground job { usually causing it to dump core and abort }
- Jobs can be moved to background using the **bg** builtin
- Jobs can be moved to foreground using the **fg** builtin
- Any signal can be sent to a process/job using the **kill** builtin { More on it in a moment }
- The **disown** builtin make the shell “forget” a job

Job control: A graphical overview



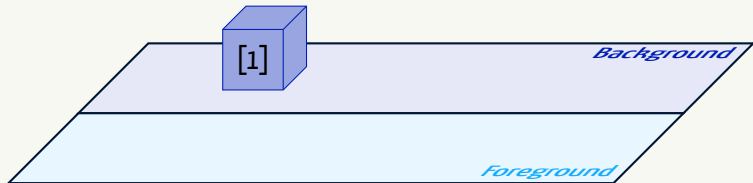
```
26 $ emacs Loops.tex
27 # Process in foreground, shell not usable
```

Job control: A graphical overview



```
26 $ emacs Loops.tex
27 # Process in foreground, shell not usable
28 # -> suspend via CTRL-Z
29 # You get your shell back, but emacs is blocked
30 [1]+  Stopped                  emacs Loops.tex
```


Job control: A graphical overview



```
26 $ emacs Loops.tex
27 # Process in foreground, shell not usable
28 # -> suspend via CTRL-Z
29 # You get your shell back, but emacs is blocked
30 [1]+  Stopped                  emacs Loops.tex
31 $ bg
32 # Alternative syntax:
33 #   bg %1      <- Using the jobnumber
34 #   bg %+     <- The last started in bg, or suspended from fg
35 #   bg %%     <- Equivalent to %+ (default if no job specified)
36 #   bg %ema   <- Job whose command begins with 'ema'
37 #   bg %?mac  <- Job whose command contains 'mac'
38 # It is an error if there is more than one matching job!
39 [1]+  emacs Loops.tex &
```

Handling processes in a script

Handling processes in a script



Handling processes in a script

- What we have discussed so far is very handy in interactive sessions
- In a script, job control is turned off by default and you will hardly need it!
- What is sometimes needed is to run processes in background
- In such a case, it is important to be aware of
 - what the `kill` builtin does
 - what is the `wait` builtin meant for
 - how to handle processes within a script → use of `$!`

Handling processes in a script

- What we have discussed so far is very handy in interactive sessions
- In a script, job control is turned off by default and you will hardly need it!
- What is sometimes needed is to run processes in background
- In such a case, it is important to be aware of
 - what the `kill` builtin does
 - what is the `wait` builtin meant for
 - how to handle processes within a script → use of `$!`
- Even **more important**, you have to understand **the parent-child relationship!**



Handling processes in a script: PIDs and parents

- In UNIX, processes are identified by a number called a PID (for Process Identifier)
- Each **running** process has a unique identifier
- Do not assume anything about PIDs: For all intents and purposes, they **are random!**
- Each UNIX process also has a parent process which is the process that started it.
- The parent process can change to the `init` process if the parent process ends before the new process does → i.e. `init` will pick up orphaned processes

Handling processes in a script: PIDs and parents

- In UNIX, processes are identified by a number called a PID (for Process Identifier)
- Each **running** process has a unique identifier
- Do not assume anything about PIDs: For all intents and purposes, they **are random!**
- Each UNIX process also has a parent process which is the process that started it.
- The parent process can change to the `init` process if the parent process ends before the new process does → i.e. `init` will pick up orphaned processes

System processes

A process PID will **never** be freed up for use after the process dies until the parent process waits for the PID to see whether it ended and retrieve its exit code. If the parent ends, the process is returned to `init`, which does this for you. This is important for one major reason: **if the parent process manages its child process, it can be absolutely certain that, even if the child process dies, no other new process can accidentally recycle the child process PID until the parent process has waited for that PID and noticed the child died.** This gives the parent process the guarantee that the PID it has for the child process will **always** point to that child process, whether it is alive or a “zombie”. Nobody else has that guarantee.

Greg's Wiki

Handling processes in a script: PIDs and parents

- In UNIX, processes are identified by a number called a PID (for Process Identifier)
- Each **running** process has a unique identifier
- Do not assume anything about PIDs: For all intents and purposes, they **are random!**
- Each UNIX process also has a parent process which is the process that started it.
- The parent process can change to the `init` process if the parent process ends before the new process does → i.e. `init` will pick up orphaned processes

System processes

A process PID will **never** be freed up for use after the process ends until the parent process waits for the PID to see whether it ended and returned `exit`. If the parent process is returned to `init`, which does not wait for the child process, the PID is not freed up. This is a process management issue. If a process dies, no other process can reuse the PID until the parent process has waited for it. The parent process has the guarantee that the PID will be freed up. It can point to that child process, whether it is alive or a "zombie". No one else has that guarantee.

Not true in the shell!

Greg's Wiki

Handling processes in a script: PIDs and parents

- In UNIX, processes are identified by a number called a PID (for Process Identifier)
- Each **running** process has a unique identifier
- Do not assume anything about PIDs: For all intents and purposes, they **are random!**
- Each UNIX process also has a parent process which is the process that started it.
- The parent process can change to the `init` process if the parent process ends before the new process does → i.e. `init` will pick up orphaned processes



What happens in the shell

Shells aggressively reap their child processes and store the exit status in memory, where it becomes available to your script upon calling `wait`. But because the child might have already been reaped before you call `wait`, there is no zombie to hold the PID.

The kernel is free to reuse that PID, and your guarantee has been violated!

Greg's Wiki

We will see an explicit example in few slides

A bit of syntax: Retrieving a process PID

- The `$!` special parameter holds the PID of the most recently executed background job

```
#!/usr/bin/env bash

#...
command &
pid=$!
# ...
```

- You cannot reliably determine when or how a process was started purely from its identifier number (PID): **do not make any assumption!**
- As you might know, there are some metadata stored together with the PID, such as the process name and the issued command to start the process. **Never rely on those!**

Stay away from parsing the process tree!

UNIX comes with a set of handy tools, among which is `ps`. This is a very helpful utility that you can use from the command line to get an overview of what processes are running.

However, `ps` output is unpredictable, highly OS-dependent, and not built for parsing!

Do not parse it in shell scripts! Ever!

A bit of syntax: The `wait` builtin

```
wait [-fn] [-p var] [jobspec or pid ...]
```

- `wait` waits until the child process specified by each process ID `pid` or job specification `jobspec` exits and return the exit status of the last command waited for
 - If no arguments are given, all currently active child processes are waited for, and the return status is zero
 - `wait -n` waits for a single job to terminate and returns its exit status
 - If not an active child process of the shell is passed to `wait`, the return status is **127**
 - `wait -p aVar` (since `Bash v5.1`) sets `aVar` to the process or job ID of the job for which the exit status is returned. The variable will be unset before any assignment.
 - ↳ **This is useful only when the `-n` option is supplied!**
-
- If a job specification is given, all processes in the job are waited for
 - Supplying the `-f` option, when job control is enabled, forces `wait` to wait for each `pid` or `jobspec` to terminate before returning its status, instead of returning when it changes status

A bit of syntax: The `wait` builtin

```
wait [-fn] [-p var] [jobspec or pid ...]
```

- `wait` waits until the child process specified by each process ID `pid` or job specification `jobspec` exits and return the exit status of the last command waited for
- If no arguments are given, all currently active child processes are waited for, and the return status is zero
- `wait -n` waits for a single job to terminate and returns its exit status
- If not an active child process of the shell is passed to `wait`, the return status is 127

```
# In a Bash script
date
{ sleep 1; exit 1; } & { sleep 3; exit 2; } &
{ sleep 5; exit 3; } & { sleep 7; exit 4; } &
wait # All currently active child processes are waited for
date
```

```
$ ./script-above.bash
Wed  4 Sep 16:13:27 CEST 2019
Wed  4 Sep 16:13:34 CEST 2019
```

A bit of syntax: The `wait` builtin

```
wait [-fn] [-p var] [jobspec or pid ...]
```

- `wait` waits until the child process specified by each process ID `pid` or job specification `jobspec` exits and return the exit status of the last command waited for
- If no arguments are given, all currently active child processes are waited for, and the return status is zero
- `wait -n` waits for a single job to terminate and returns its exit status
- If not an active child process of the shell is passed to `wait`, the return status is 127

```
# In a Bash script
date; sleep 3 & pid=$!
echo "PID=${pid}"; ps -p ${pid}; wait ${pid}; date
```

```
$ ./script-above.bash
Wed  4 Sep 16:17:19 CEST 2019
PID=11353
  PID TTY          TIME CMD
11353 pts/11    00:00:00 sleep
Wed  4 Sep 16:17:22 CEST 2019
```

A bit of syntax: The `wait` builtin

```
wait [-fn] [-p var] [jobspec or pid ...]
```

- `wait` waits until the child process specified by each process ID `pid` or job specification `jobspec` exits and return the exit status of the last command waited for
- If no arguments are given, all currently active child processes are waited for, and the return status is zero
- `wait -n` waits for a single job to terminate and returns its exit status
- If not an active child process of the shell is passed to `wait`, the return status is 127

```
# In a Bash script (-n option since Bash v4.3)
{ sleep 1; exit 1; } & { sleep 3; exit 2; } &
{ sleep 2; exit 3; } &
for f in {1..3}; do
    wait -n; echo "wait return status: $? at $(date +%X)"
done
```

```
$ ./script-above.bash
wait return status: 1 at 16:36:16
wait return status: 3 at 16:36:17
wait return status: 2 at 16:36:18
```

A bit of syntax: The `wait` builtin

```
wait [-fn] [-p var] [jobspec or pid ...]
```

- `wait` waits until the child process specified by each process ID `pid` or job specification `jobspec` exits and return the exit status of the last command waited for
- If no arguments are given, all currently active child processes are waited for, and the return status is zero
- `wait -n` waits for a single job to terminate and returns its exit status
- If not an active child process of the shell is passed to `wait`, the return status is 127

```
$ wait 1234
bash: wait: pid 1234 is not a child of this shell
$ echo $?
127
```

A bit of syntax: The `kill` builtin

```
kill [-s sigspec] [-n signum] [-sigspec] jobspec or pid
```

- Despite its name, the `kill` command sends a signal to a process
- There are different way to specify a signal, use what you prefer
- If no signal is specified, `SIGTERM` (15) is sent

```
$ kill -l
```

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP
21) SIGTTIN	22) SIGTOU	23) SIGURG	24) SIGXCPU	25) SIGXFSZ
26) SIGVTALRM	27) SIGPROF	28) SIGWINCH	29) SIGIO	30) SIGPWR
31) SIGSYS	34) SIGRTMIN	35) SIGRTMIN+1	36) SIGRTMIN+2	37) SIGRTMIN+3
38) SIGRTMIN+4	39) SIGRTMIN+5	40) SIGRTMIN+6	41) SIGRTMIN+7	42) SIGRTMIN+8
43) SIGRTMIN+9	44) SIGRTMIN+10	45) SIGRTMIN+11	46) SIGRTMIN+12	47) SIGRTMIN+13
48) SIGRTMIN+14	49) SIGRTMIN+15	50) SIGRTMAX-14	51) SIGRTMAX-13	52) SIGRTMAX-12
53) SIGRTMAX-11	54) SIGRTMAX-10	55) SIGRTMAX-9	56) SIGRTMAX-8	57) SIGRTMAX-7
58) SIGRTMAX-6	59) SIGRTMAX-5	60) SIGRTMAX-4	61) SIGRTMAX-3	62) SIGRTMAX-2
63) SIGRTMAX-1	64) SIGRTMAX			

```
$ kill -l 15
```

```
TERM
```


A bit of syntax: The `kill` builtin

```
kill [-s sigspec] [-n signum] [-sigspec] jobspec or pid
```

- Despite its name, the `kill` command sends a signal to a process
- There are different way to specify a signal, use what you prefer
- If no signal is specified, SIGTERM (15) is sent

`man 2 kill`

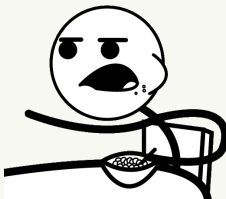
If signal is 0, then no signal is sent, but existence and permission checks are still performed; this can be used to check for the existence of a process ID or process group ID that the caller is permitted to signal.

```
1 # Waiting for a process to end
2 # (when you have permissions to send signals)
3 while kill -0 "${pid}"; do
4     sleep 1
5 done
```

A potentially catastrophic hidden danger

A potentially catastrophic hidden danger

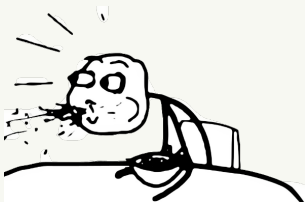
```
1  #!/usr/bin/env bash
2                                     #  EXAMPLE 1: wait
3  ( exit 12 ) &
4  pid=$!
5  while { sleep 0 & [[ "$!" != "${pid}" ]]; }; do
6      :
7  done
8  wait "$pid"
9  echo "$?"
```



A potentially catastrophic hidden danger

```
1  #!/usr/bin/env bash
2                                     #  EXAMPLE 1: wait
3  ( exit 12 ) &
4  pid=$!
5  while { sleep 0 & [[ "$!" != "${pid}" ]]; }; do
6      :
7  done
8  wait "$pid"
9  echo "$?"
```

```
$ ./script-above.bash
0  # Yes, 0 and not 12... WHAAAAAT?
```



A potentially catastrophic hidden danger

```
1  #!/usr/bin/env bash
2                                     #  EXAMPLE 1: wait
3  ( exit 12 ) &
4  pid=$!
5  while { sleep 0 & [[ "$!" != "${pid}" ]]; }; do
6      :
7  done
8  wait "$pid"
9  echo "$?"

$ ./script-above.bash
0  # Yes, 0 and not 12... WHAAAAAT?
```

Sad but true!

The way shells are programmed, as soon as a child process dies, the shell will call `wait()` on it right away (storing the termination status as part of its internal state), which will free the PID for reuse by another process.

A potentially catastrophic hidden danger

```
10 #!/usr/bin/env bash
11                               # EXAMPLE 2: kill
12 long_running_command &
13 pid=$!
14 echo "Killing long_running_command on PID ${pid} in 24h"
15 sleep 86400
16 echo 'Time up!'
17 kill "${pid}"
```

Who will receive the SIGTERM signal?!

Sad but true!

The way shells are programmed, as soon as a child process dies, the shell will call `wait()` on it right away (storing the termination status as part of its internal state), which will free the PID for reuse by another process.

I absolutely need to know if the PID is the right one

What should I do?

I absolutely need to know if the PID is the right one

What should I do?

Do not use Bash!

Yes, implement the code launching a background process in C or Python, Perl, Ruby, etc. **Not in shell.** Those will not have this problem, since they won't reap children by default (like the shell does) and you will have to do it explicitly there. Or consider launching background processes using a system manager, such as `systemd`.

I absolutely need to know if the PID is the right one

What should I do?

Do not use Bash!

Yes, implement the code launching a background process in C or Python, Perl, Ruby, etc. **Not in shell.** Those will not have this problem, since they won't reap children by default (like the shell does) and you will have to do it explicitly there. Or consider launching background processes using a system manager, such as `systemd`.

However

Note that the kernel will typically try hard to avoid reusing PIDs, at least try to delay reusing a PID, exactly because in some cases there are no guarantees that the PID hasn't been reused, so the kernel tries to minimise this situation where a signal will be delivered to the wrong process.

I absolutely need to know if the PID is the right one

What should I do?

Do not use Bash!

Yes, implement the code launching a background process in C or Python, Perl, Ruby, etc. **Not in shell**. Those will not have this problem, since they won't reap children by default (like the shell does) and you will have to do it explicitly there. Or consider launching background processes using a system manager, such as `systemd`.

Alternatively

You can save the start time of the original process and, before using it, check that the start time of the process with that PID matches what you saved. The pair PID, start-time is a unique identifier for the processes in Linux. However, this does not solve the issue here.

```
$ sleep 10 &
[1] 4451
$ ps -p 4451 -h -o lstart
Thu Sep  5 18:22:43 2019
```

A work-around for exit codes

- If you have several (maybe long) processes in background during the execution of a script and you are interested in their exit code, you can write them to a file
- Alternatively, a possible idea is to use redirection faking a file and taking advantage of the fact that **in Bash the processes inside the process substitution are not waited for**

```
1  #!/usr/bin/env bash
2  {
3      command_pid=$!
4      # ...
5      another_command &
6      # ...
7      read <&3 exitCode
8      if [[ "${exitCode}" -eq 0 ]]; then
9          echo "command was successful!"
10     fi
11 } 3< <(command > logfile 2>&1; echo "$?")
```

A work-around for exit codes

- If you have several (maybe long) processes in background during the execution of a script and you are interested in their exit code, you can write them to a file
- Alternatively, a possible idea is to use redirection faking a file and taking advantage of the fact that **in Bash the processes inside the process substitution are not waited for**

```
12 #!/usr/bin/env bash
13 {
14     sleep_pid=$!
15     while { ( exit 12 ) & [[ "$!" != "${sleep_pid}" ]]; }; do
16         :
17     done
18     wait "${sleep_pid}"
19     waitCode=$?
20     read <&3 exitCode
21     echo "Command in process substitution: ${exitCode}"
22     echo "Command in while loop: ${waitCode}"
23 } 3< <(sleep 1; echo "$?")
```

A work-around for exit codes

- If you have several (maybe long) processes in background during the execution of a script and you are interested in their exit code, you can write them to a file
- Alternatively, a possible idea is to use redirection faking a file and taking advantage of the fact that **in Bash the processes inside the process substitution are not waited for**

```
12 #!/usr/bin/env bash
13 {
14     sleep_pid=$!
15     while { ( exit 12 ) & [[ "$!" != "${sleep_pid}" ]]; }; do
16         :
17     done
18     wait "${sleep_pid}"
19     waitCode=$?
20     read <&3 exitCode
21     echo "Command in process substitution: ${exitCode}"
22     echo "Command in           while loop: ${waitCode}"
23 } 3< <(sleep 1; echo "$?")
```

```
$ ./script-above.bash
Command in process substitution: 0
Command in           while loop: 12
```

Process Management: Conclusions

- It is a potentially tough world: Be scared but not too much
- Bash might not be the most appropriate tool
- Be aware of the truth (what we just discussed)
- If you want to have something running in background, why do you want so?
- There are tools which you might consider:
 - 1 `systemd` (system manager)
 - 2 `timeout` (run a command with a time limit)
 - 3 `xargs -P` or `parallel` (to run independent tasks in parallel)

Process Management: Conclusions

- It is a potentially tough world: Be scared but not too much
- Bash might not be the most appropriate tool
- Be aware of the truth (what we just discussed)
- If you want to have something running in background, why do you want so?
- There are tools which you might consider:
 - 1 `systemd` (system manager)
 - 2 `timeout` (run a command with a time limit)
 - 3 `xargs -P` or `parallel` (to run independent tasks in parallel)

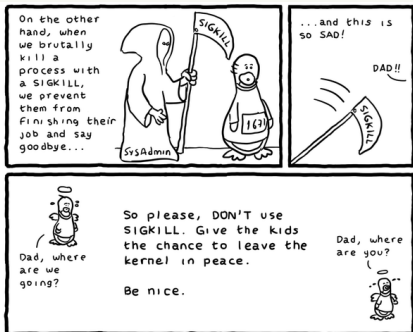
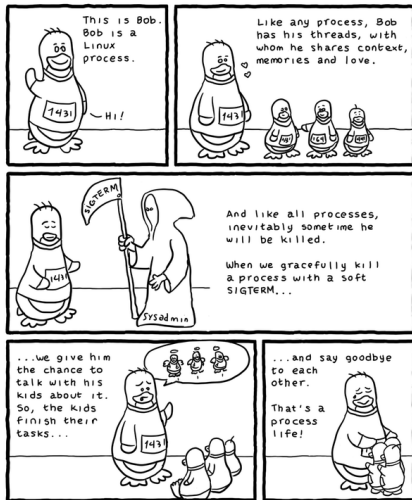
About using `kill -9`

Do not use `kill -9`, ever. For any reason. **Unless you wrote the program to which you're sending the SIGKILL, and know that you can clean up the mess it leaves.** Because you're debugging it. If a process is not responding to normal signals, it's probably in "state D" (as shown on `ps u`), which means it's currently executing a system call. If that's the case, you're probably looking at a dead hard drive, or a dead NFS server, or a kernel bug, or something else along those lines.

And you won't be able to kill the process anyway, SIGKILL or not.

Greg's Wiki

Process Management: Conclusions



Daniel Storr (turnoff.ws)

Let's practice!

Concurrency execution of tests

As always, start from your previous code (or the last sample solution):

- 1 Modify your tests to `sleep` a given amount of seconds.
- 2 Add a `-j` command line option with an integer as argument { how many tests your runner will concurrently run }
- 3 Implement the logic to concurrently run tests in `Run_Tests`*.
- 4 Test your main script with different values of the `-j` option.
- 5 Duplicate your fake tests to have at least `$( ( $(nproc)*2 ) )`.
- 6 Use `time` to see what happens varying `-j` value.
- 7 **Optional:** Change `sleep` to the following: { There is a dedicated exercise in the sheet 4 to deepen into this task. }

```
function Burn_Cpu()
{
    local iterations=${1:-500000} x=0
    for ((i=0; i<iterations; i++)); do
        (( x += i ))
    done
}
```

What do you observe?

Traps



Moulin on the Breiðamerkurjökull

Motivation

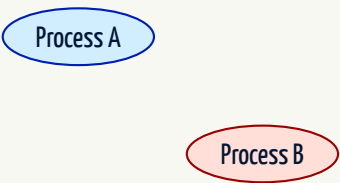
Consider the following script (**not nice code**):

```
1  #!/usr/bin/env bash
2  function CreateAuxiliaryFiles(){
3      # ...
4  }
5  function CleanAuxiliaryFiles(){
6      # ...
7  }
8  CreateAuxiliaryFiles
9  gnuplot "${temporaryFileToPlot}"
10 if [[ "${savePlot}" = 'TRUE' ]]; then
11     pdflatex "${outputFilename}.tex"
12 fi
13 CleanAuxiliaryFiles
14 exit 0
```

What happens if the script is terminated by the user, e.g. via CTRL-C?

Signal handlers or traps

- As you know you can send signals to processes via the `kill` builtin
- However, in shell scripts, **it is possible to associate a behaviour upon receiving a signal!**
- If you think about it, it is a very powerful technique
- In order to perform an action when a signal is received, a trap has to be set up

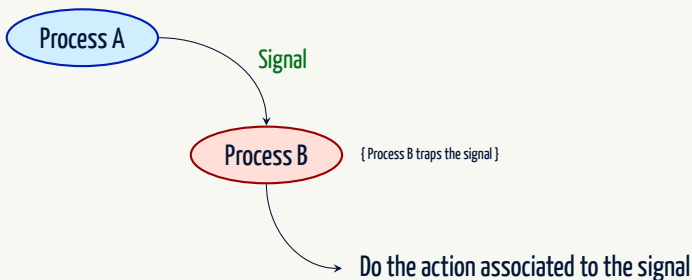


Process A

Process B

Signal handlers or traps

- As you know you can send signals to processes via the `kill` builtin
- However, in shell scripts, **it is possible to associate a behaviour upon receiving a signal!**
- If you think about it, it is a very powerful technique
- In order to perform an action when a signal is received, a trap has to be set up



Signal handlers or traps

```
trap 'some code' signal(s)
```

Using this form, a signal handler is set up for each signal in the list

When one of these signals is received, the commands in the first argument will be executed

```
trap '' signal(s)
```

Using this form, each signal in the list will be ignored. **Most scripts should not do this!**

```
trap - signal(s)
```

Using this form, each signal in the list will be restored to its default behaviour

```
trap signal
```

Legacy syntax: As the previous but for one signal only! { Prefer the previous, slightly more explicit syntax }

```
trap
```

With no arguments, print a list of signal handlers

Common used signals

HUP Hang Up. The controlling terminal has gone away.

INT Interrupt. The user has pressed the interrupt key (usually **Ctrl-C** or **DEL**).

QUIT Quit. The user has pressed the quit key (usually **Ctrl-**). Exit and dump core.

KILL Kill. **This signal cannot be caught or ignored.** Unconditionally fatal. **No cleanup possible.**

TERM Terminate. This is the default signal sent by the `kill` builtin.

EXIT Not really a signal. In a Bash script (non-interactive), an EXIT trap is run on any exit, signalled or not. In other POSIX shells only when the shell process exits.

Remember

If you are asking a program to terminate, you should always use SIGTERM. This will give the program a chance to catch the signal and clean up. If you use SIGKILL, the program cannot clean up, and **may leave files in a corrupted state.**

A simple example

Common use case

Traps can be set up to intercept a fatal signal, perform cleanup, and then exit gracefully.

```
1  #!/usr/bin/env bash
2  temporaryFile=$(mktemp) || exit
3  trap 'rm -f "${temporaryFile}"' EXIT
```

Use a function whenever you need to achieve a more complex task

```
5  #!/usr/bin/env bash
6  function CleanAuxiliaryFiles(){ # If you trap INT (CTRL-C)
7      # ...                        # do not forget to exit unless
8  }                                # you have reasons not to do so
9  trap 'CleanAuxiliaryFiles' EXIT
```

Only in Bash!

The special name EXIT is preferred for any signal handler that simply wants to clean up upon exiting. So to clean up, just trap EXIT and call a cleanup function from there. **Don't trap a bunch of signals.**

When is the signal exactly handled?

A subtle but important feature

When Bash is executing an external command in the foreground, it does not handle any signals received until the foreground process terminates.

```
1 #!/usr/bin/env bash
2 echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
3 trap 'echo "Doing some cleaning before exiting!"' EXIT
4 echo "Wait 1h"
5 sleep 3600
```

- If you kill the script using `kill -s INT` from another terminal (not with CTRL-C), Bash will wait for `sleep` to exit before calling the trap
→ **That's probably not what you expect!**

When is the signal exactly handled?

A subtle but important feature

When Bash is executing an external command in the foreground, it does not handle any signals received until the foreground process terminates.

```
6  #!/usr/bin/env bash
7  echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
8  trap 'echo "Doing some cleaning before exiting!"' EXIT
9  echo "Wait 1h"; sleep 3600 & wait $!
10 # Note that sleep 3600 will not be killed and will
11 # continue to run when you send a INT signal!
```

- If you kill the script using `kill -s INT` from another terminal (not with CTRL-C), Bash will wait for `sleep` to exit before calling the trap
→ **That's probably not what you expect!**
- A work-around is to use a **builtin** that will be interrupted, such as `wait`
- Any Bash internal command will be interrupted by a (non-ignored) incoming signal!

When is the signal exactly handled?

A subtle but important feature

When Bash is executing an external command in the foreground, it does not handle any signals received until the foreground process terminates.

```
12 #!/usr/bin/env bash
13 echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
14 unset -v pid
15 trap 'echo "Cleaning!"; [[ $pid ]] && kill "${pid}"' EXIT
16 echo "Wait 1h"
17 sleep 3600 & pid=$!; wait
```

- If you kill the script using `kill -s INT` from another terminal (not with CTRL-C), Bash will wait for `sleep` to exit before calling the trap
→ **That's probably not what you expect!**
- A work-around is to use a **builtin** that will be interrupted, such as `wait`
- Any Bash internal command will be interrupted by a (non-ignored) incoming signal!
- If you want the background job to be killed when the script is killed, add that to the trap!

What indeed is CTRL-C doing?

Let us consider again the previous example

```
#!/usr/bin/env bash

echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
trap 'echo "Doing some cleaning before exiting!"' EXIT
echo "Wait 1h"; sleep 3600
```

- As said, sending to this script the INT signal from another terminal does not terminate it
- However, CTRL-C does terminate it... but why?

What indeed is CTRL-C doing?

Let us consider again the previous example

```
#!/usr/bin/env bash

echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
trap 'echo "Doing some cleaning before exiting!"' EXIT
echo "Wait 1h"; sleep 3600
```

- As said, sending to this script the INT signal from another terminal does not terminate it
- However, CTRL-C does terminate it... but why?
- Because **processes are organised in groups**:
 - The leader process (i.e. the process that created the group)
 - Any other process started by the leader process
- **CTRL-C sends the INT signal to the entire process group**
- Terminals keep track of the foreground process group: When receiving a CTRL-C, they send the SIGINT to the entire foreground group!

What indeed is CTRL-C doing?

Let us consider again the previous example

```
#!/usr/bin/env bash

echo "My PID is $$ and my PGID is $(ps -h -p $$ -o pgid)"
trap 'echo "Doing some cleaning before exiting!"' EXIT
echo "Wait 1h"; sleep 3600
```

- As said, sending to this script the INT signal from another terminal does not terminate it
- However, CTRL-C does terminate it... but why?
- Because **processes are organised in groups**:
 - The leader process (i.e. the process that created the group)
 - Any other process started by the leader process
- **CTRL-C sends the INT signal to the entire process group**
- Terminals keep track of the foreground process group: When receiving a CTRL-C, they send the SIGINT to the entire foreground group!

```
kill -s INT -123 # will kill the process group with the ID 123
```

Trapping SIGINT and SIGQUIT: Be careful!

- Bash is among a few shells that implement a **wait and cooperative exit** approach at handling SIGINT/SIGQUIT delivery
- When interpreting a script, upon receiving a SIGINT,
 - 1 it doesn't exit straight away, instead
 - 2 it waits for the currently running command to return and
 - 3 it only exits – **by killing itself with SIGINT** – if that command was also killed by that SIGINT
- The idea is that, if your script calls `vi` for instance, and you press CTRL-C within `vi` to cancel an action, that should not be considered as a request to abort the script

What does it mean?

Imagine you're writing a script and that script exits normally upon receiving SIGINT.

That means that if that script is invoked from another Bash script,

CTRL-C will no longer interrupt that other script!

Trapping SIGINT and SIGQUIT: Be careful!

- Bash is among a few shells that implement a **wait and cooperative exit** approach at handling SIGINT/SIGQUIT delivery
- When interpreting a script, upon receiving a SIGINT,
 - 1 it doesn't exit straight away, instead
 - 2 it waits for the currently running command to return and
 - 3 it only exits – by killing itself with SIGINT – if that command was also killed by that SIGINT

The `ping` command returns with 0 when host is reachable (the ping has been answered) and non-zero otherwise when interrupted (which is the only way for ping to return in that case).

```
1 #!/usr/bin/env bash
2 for index in {1..254}; do
3     ping -c 2 "192.168.1.${index}"
4 done
```

CTRL-C will send a INT signal to the script, but since `ping` just returns 0 or 1 and is not killed by SIGINT, only the **current ping** will terminate and the **for** loop will continue!

Trapping SIGINT and SIGQUIT: Be careful!

- Bash is among a few shells that implement a **wait and cooperative exit** approach at handling SIGINT/SIGQUIT delivery
- When interpreting a script, upon receiving a SIGINT,
 - 1 it doesn't exit straight away, instead
 - 2 it waits for the currently running command to return and
 - 3 it only exits – by killing itself with SIGINT – if that command was also killed by that SIGINT

Commands that don't have a SIGINT handler (like `sleep`) or **do the right thing of killing themselves with SIGINT upon receiving SIGINT** (like `bash` itself does) don't exhibit the problem!

```
5  #!/usr/bin/env bash
6  index=1
7  while [[ "${index}" -le 100 ]]; do
8      printf "%d " "$((index++))"
9      sleep 1
10  done; echo # This script terminates correctly via CTRL-C
```

Trapping SIGINT and SIGQUIT: Be careful!

- Bash is among a few shells that implement a **wait and cooperative exit** approach at handling SIGINT/SIGQUIT delivery
- When interpreting a script, upon receiving a SIGINT,
 - 1 it doesn't exit straight away, instead
 - 2 it waits for the currently running command to return and
 - 3 it only exits – **by killing itself with SIGINT** – if that command was also killed by that SIGINT

Take-home lesson

If you choose to set up a handler for SIGINT (rather than using the EXIT trap), you should be aware that **a process that exits in response to SIGINT should kill itself with SIGINT rather than simply exiting**, to avoid causing problems for its caller. The same goes for SIGQUIT.

```
trap 'DoYourStuffHere; trap - INT; kill -s INT "$$" ' INT
```

Exercise sheet 4 (I)

Some guidance about the session

- You have roughly 60 minutes
 - ↳ You won't manage to do everything! { That's not expected either! }
- Each exercise should be solved in a dedicated script { Use the terminal to experiment, if needed }
- For the moment you can only tackle exercises 1, 2 and 5
- Start with exercise 2 but after 30 minutes switch to either exercise 1 or 5
 - ↳ Exercise 1 is more about `wait` and possibly HPC oriented
 - ↳ Exercise 5 is more about understanding concurrency
 - ↳ It would be instructive to try to do both { if not now in the next session }
- Note down your questions, doubts, curiosities, ...
- If you can't resist: Give your solution to an AI to get feedback { The trainer can also give you feedback }

Come up with suggestions! —

In the closing sessions we'll solve an exercise together.
If you have any preference, feel free to speak up.

Error handling



How the Breiðamerkurjökull shrank from 2017 to 2018

Error handling in the shell

Naturally an extremely difficult task

The goal of automatic error detection is a noble one, but it requires the ability to tell when an error actually occurred. In modern high-level languages, most tasks are performed by using the language's builtin commands or features. The language knows whether (for example) you tried to divide by zero, or open a file that you can't open, and so on. It can take action based on this knowledge.

But in the shell, most of the tasks you actually care about are done by external programs. The shell can't tell whether an external program encountered something that it considers an error – and even if it could, it wouldn't know whether the error is an important one, worthy of aborting the entire program, or whether it should carry on.

Greg's Wiki

- If you come from a language like C, you might be used to it.
- If you come from a language like Python or C++, do not be frustrated.

What does Bash do when an error occurs?

I.e. when a command has a non-zero exit code!

The default behaviour

An error is printed and **execution continues**.

```
1  #!/usr/bin/env bash
2
3  # A test.bash script
4  echo "Before error"
5  notExistingCommand
6  echo "After error"    # <-- This line is executed!
```

```
$ ./test.bash
Before error
./test.bash: line 5: notExistingCommand: command not found
After error
```


What does Bash do when an error occurs?

I.e. when a command has a non-zero exit code!

The default behaviour

An error is printed and **execution continues**.

Do not forget it!

Things that we consider an error are not necessarily an error for Bash!

```
$ invisibleVariable="I am magic"
$ echo "${invisibleVariable}"
--
```

What does Bash do when an error occurs?

I.e. when a command has a non-zero exit code!

The default behaviour

An error is printed and **execution continues**.

Do not forget it!

Things that we consider an error are not necessarily an error for Bash!

We clearly want more than this!

- 1 We often need to react on error and avoid execution to continue.
- 2 It might be crucial to prevent undefined variables to be used!

```
# About 1: Somewhere in a script
cd NotExistingFolder
rm -r *    # AAAAAAARGH!
```

What does Bash do when an error occurs?

I.e. when a command has a non-zero exit code!

The default behaviour

An error is printed and **execution continues**.

Do not forget it!

Things that we consider an error are not necessarily an error for Bash!

We clearly want more than this!

- 1 We often need to react on error and avoid execution to continue.
- 2 It might be crucial to prevent undefined variables to be used!

```
#!/usr/bin/env bash
# About 2: And if you call this script without arguments!?
pathPrefix="$1"
rm -r "${pathPrefix}/bin"    # AAAAAAARGH!
```

Exit codes are the only available ingredient

...the rest is basically up to you!

Purist approach

Check and handle errors properly testing exit codes **when needed**.

- `$?` holds the exit code of the last command executed before any given line.
- Any executed command will terminate with an exit code that can be inspected.
- You can expect external command exit codes to be well documented.

```
# From 'man grep'
...
EXIT STATUS
  Normally the exit status is 0 if a line is selected,
  1 if no lines were selected, and 2 if an error occurred.
  However, if the -q or --quiet or --silent is used and a
  line is selected, the exit status is 0 even if an
  error occurred.
...
```

Exit codes are the only available ingredient

...the rest is basically up to you!

Purist approach

Check and handle errors properly testing exit codes **when needed**.

- `$?` holds the exit code of the last command executed before any given line.
- Any executed command will terminate with an exit code that can be inspected.
- You can expect external command exit codes to be well documented.

```
# In a script at some point
grep 'Hello' MyBook.txt
case $? in
  0 ) echo 'Found' ;;
  1 ) echo 'Not found' ;;
  2 ) echo 'Error occurred in grep!'; exit 1 ;;
  * ) echo 'Matrix changed something...'; exit 2 ;;
esac
```

Exit codes are the only available ingredient

...the rest is basically up to you!

Purist approach

Check and handle errors properly testing exit codes **when needed**.

- `$?` holds the exit code of the last command executed before any given line.
- Any executed command will terminate with an exit code that can be inspected.
- You can expect external command exit codes to be well documented.

```
# Coming back to the previous example
cd NotExistingFolder || exit 1  # Exit on failure!
rm -r *                        # Now better!
```

```
# Alternatively not exiting
if cd NotExistingFolder; then
    rm -r *
fi
```

Of course, this is just an example, you would rather write `rm NotExistingFolder/*`, right?!

Do I have to add `|| exit 1` everywhere in my scripts?

Do I have to add `|| exit 1` everywhere in my scripts?

Purist approach (again)

Check and handle errors properly testing exit codes **when needed**.

Do I have to add `|| exit 1` everywhere in my scripts?

Purist approach (again)

Check and handle errors properly testing exit codes **when needed**.

The developers of the original Bourne shell decided that they would create a feature that would allow the shell to check the exit status of every command that it runs, and abort if one of them returns non-zero. Thus, `set -e` was born. But many commands return non-zero even when there wasn't an error. [...] So the shell implementers made a bunch of special rules, like “commands that are part of an if test are immune”, and “commands in a pipeline, other than the last one, are immune”.

Greg's Wiki

The `set -o errexit` is probably the most controversial feature of Bash!

- Many articles are against using it, e.g. [Greg's Wiki](#).
- People encourage its use, e.g. [David Pashley](#).

Enabling the `-e` shell option: Let's read the manual

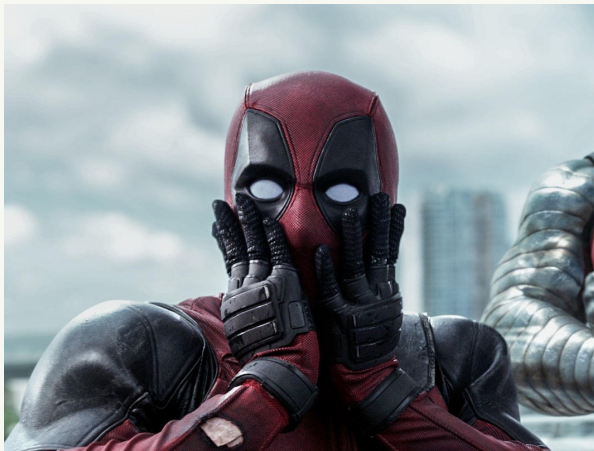
- Exit immediately if a pipeline, which may consist of a single simple command, a list, or a compound command returns a non-zero status.
- The shell does not exit if the command that fails
 - is part of the command list immediately following a `while` or `until` keyword,
 - is part of the test in an `if` statement,
 - is part of any command executed in a `&&` or `||` list except the command following the final `&&` or `||`,
 - is any command in a pipeline but the last,or if the command's return status is being inverted with `!`.
- If a compound command other than a subshell returns a non-zero status because a command failed while `-e` was being ignored, the shell does not exit.
- A trap on `ERR`, if set, is executed before the shell exits.
- This option applies to the shell environment and each subshell environment separately, and may cause subshells to exit before executing all the commands in the subshell.

Enabling the `-e` shell option: Let's read the manual

Your reaction might be like:

Enabling the `-e` shell option: Let's read the manual

Your reaction might be like:



But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- Checking and storing error codes via `$?` is not possible as before

But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- Checking and storing error codes via `$?` is not possible as before

```
1  #!/usr/bin/env bash
2  set -e
3  i=0
4  ((i++))          # ((...)) is a compound command!
5  echo "i is $i"   # This is NOT printed
```

The postfix increment operator returns the old value of the variable.

```
1  #!/usr/bin/env bash
2  set -e
3  i=1
4  ((i++))          # ((...)) is a compound command!
5  echo "i is $i"   # This is here printed
```

But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- Checking and storing error codes via `$?` is not possible as before

```
1  #!/usr/bin/env bash
2  set -e
3  i=0
4  ((i++)) || true
5  echo "i is $i"      # This is ALWAYS printed

6  i=$((i-1))          # Simple command without command subst.
7  echo "i is $i"      # This is ALWAYS printed

8  i=$(false)          # Simple command with failing $(...)
9  echo "i is $i"      # This is NOT printed
```

Simple command expansion

[...] If one of the expansions contained a command substitution, the exit status of the command is the exit status of the last command substitution performed. If there were no command substitutions, the command exits with a status of zero.

Bash manual v5.2 section 3.7.1

But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- Checking and storing error codes via `$?` is not possible as before

```
1  #!/usr/bin/env bash
2  set -e

3  function Failure() { false; }
4  function FailureMsg() {
5      false
6      echo "Failed!" >&2
7  }

8  # Command substitution does not inherit errexit!
9  echo "${Failure; echo 'A'}" # This prints 'A'

10 echo 'Begin'
11 aVar=$((FailureMsg)) # No failure because of echo in function
12 echo 'Middle'
13 aVar=$((Failure))    # Function returns non zero exit code
14 echo "End"           # This is NOT printed
```


But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- **Checking and storing error codes via `$?` is not possible as before**

```
1  #!/usr/bin/env bash
2  set -e
3  function Simulation(){ return 1; }

4  if Simulation; then          # No errexit in if-clauses
5      echo 'Success!'
6  else
7      echo 'Failure!'
8  fi

9  Simulation                   # Here the script exits!

10 if [[ $? -eq 0 ]]; then      # This if-clause is skipped!
11     echo 'Success!'
12 else
13     echo 'Failure!'
14 fi
```

But what does it mean, in practice?

- You need to **deeply** understand this shell option
- Your way of programming has to adapt to all corner cases
- Checking and storing error codes via `$?` is not possible as before

Spend some time on it

In exercise sheet 4 you will deal again in detail with some corner cases!

And if I do not want to use it?

Do I have other alternatives beyond the purist approach and enabling `errexit`?

Not really...

Consider to enable `inheriterrexit` shell option

It was introduced in Bash v4.4

If set, command substitution inherits the value of the errexit option, instead of unsetting it in the subshell environment. This option is enabled when POSIX mode is enabled.

Bash manual

```
1  #!/usr/bin/env bash
2  set -e
3  shopt -s inheriterrexit

4  function Failure() { false; }
5  function FailureMsg() {
6      false
7      echo "Failed!" >&2
8  }

9  echo "${Failure; echo 'A'}" # Now this does NOT print 'A'

10 echo 'Begin'
11 aVar=$(FailureMsg) # Now the script exits HERE!
12 echo 'Middle'
13 aVar=$(Failure)    # Function fails
14 echo "End"         # This is NOT printed
```

Trap on ERR and the erretrace shell option

The ERR signal specification

The trap on **ERR** is executed exactly when Bash would exit with the **errexit** option enabled. Said differently, the same conditions obeyed by the **errexit** option apply to traps on **ERR**.

The erretrace shell option

If set, any trap on **ERR** is inherited by shell functions, command substitutions, and commands executed in a subshell environment. The **ERR** trap is normally not inherited in such cases.

Bash manual

set -E in combination with **ERR** traps is a bit like an improved version of **set -e** in that it allows you to define your own error handling.

Stéphane Chazelas

Trap on ERR and the erretrace shell option

```
1  #!/usr/bin/env bash
2  set -E
3  trap '[[ "$?" -ne 77 ]] || exit 77' ERR
4  (
5      echo 'Here'
6      (
7          echo 'There'
8          ( exit 12 )      # not 77, exit only this subshell
9          echo 'Somewhere'
10         exit 77         # exit all subshells
11     )
12     echo 'Not here'
13 )
14 echo 'Not here either'
```

```
$ ./previous-script.bash
Here
There
Somewhere
```

Enabling the `-u` shell option

`set -o nounset`

Treat unset variables and parameters other than the special parameters `@` or `*` as an error when performing parameter expansion. An error message will be written to the standard error, and a non-interactive shell will exit.

Bash manual

```
$ echo "_${unsetVariable}_"
--
$ set -u; echo "_${unsetVariable}_"      # A script would
bash: unsetVariable: unbound variable # exit here!
$ echo "_${unsetVariable-}_" ; set +u
--
```

Few (minor) drawbacks exist

- 1 Turning `nounset` on in a correct existing script might still break it.
- 2 If some code is not executed, `set -u` will have no effect on it either.
- 3 Versions of Bash prior to v4.4 [handle the `\[@\]`-expansion of empty arrays differently](#).

Is the glass half full or half empty?

A reasonable person might say: _____

I don't want to rely on an unreliable feature. I understood the behaviour of the shell when an error occurs and I can do error handling myself.

Is the glass half full or half empty?

A reasonable person might say:

I don't want to rely on an unreliable feature. I understood the behaviour of the shell when an error occurs and I can do error handling myself.

Another reasonable person might say:

Yes, *errexit* is not a perfect feature of Bash, but it has useful semantics, so I don't want to exclude it from the toolbox. Still I have to work a bit to get acquainted with all corner cases, but I think it is worth the effort. Especially because I do not want to be bitten by mistakes in doing error handling by hand. Hopefully, future versions of Bash will improve this aspect.

Is the glass half full or half empty?

A reasonable person might say:

I don't want to rely on an unreliable feature. I understood the behaviour of the shell when an error occurs and I can do error handling myself.

Another reasonable person might say:

Yes, *errexit* is not a perfect feature of Bash, but it has useful semantics, so I don't want to exclude it from the toolbox. Still I have to work a bit to get acquainted with all corner cases, but I think it is worth the effort. Especially because I do not want to be bitten by mistakes in doing error handling by hand. Hopefully, future versions of Bash will improve this aspect.

Choose your way

Probably consistency within a script is the most important aspect. My recommendation is to **explore this area of Bash in all ways before taking your decision.**

Let's practice!

Error handling and final polish

Pause to think

It should be by now clear that we want to inspect exit codes in our test framework to determine success or failure. Therefore `set -e` is not an option in test code. However, what about the codebase under test? In test code you will usually use functionality from your codebase (typically calling functions). A trivial statement is that you want to execute such a code as it is usually executed in your codebase. So you might want to run code enabling exit on error from a script that does not want it. **In this exercise we want to test code using:**

```
set -e; shopt -s inheriterrexit
```

As warm-up exercise

As always, start from your previous code (or the last sample solution):

- 1 Enable `set -u` in your script and check if it still runs
↳ Investigate and fix possible failures

Error handling and final polish

- 2 Pretend to have in your codebase a function `Is_Odd` that you want to test as unit

```
function Is_Odd() {  
    if [[ ! $1 =~ ^([0-9]*)$ ]]; then  
        exit 42  
    elif (( $1 % 2 == 0 )); then return 1; else return 0; fi  
}
```

- 3 Write a test file that sources that unit of your codebase and tests it.

- How do you call the function?
- How do you avoid such a function exit your test?
- How do you guarantee to run with the codebase Bash behaviour?
- Can you use any of the following?

```
Is_Odd 666 && exit 1  
if ! Is_Odd 17; then  
    exit 1  
fi
```

- 4 Encapsulate the codebase function call in a dedicated function.

Error handling and final polish

Optional and advanced:

Change your framework to treat functions with a given name in test files as tests. Running a test does not mean any more running a script file, but rather executing a function. Tests files are then made by functions only and can be sourced by your runner. You have to decide on function prefixes and stick to them. For example, use `Unit_Test__` for unit tests and similar ones for the other types.

4 Implement a `Define_Available_Tests` in order to

- locate the tests files;
- source them;
- parse them to extract function names.

Hint: You need to be generic in extracting the function name:

```
# This assumes that the opening brace { of the function is on a new line
local -r \
    grep_regex='^function[[:space:]]+''${prefix}''[-[:alnum:]_.:]+\(\)[[:space:]]*'$
```

Sed (and Awk)



A double wave

Why do we need them?

- In science, most of our time in the terminal is spent acting on files
- Mastering file handling in general can allow us to simplify the data analysis software
- Typical operations might be
 - Prepare data for plotting in a given format
 - Search patterns in a file
 - Extract portion of a file
 - Transform a file (e.g. remove trailing spaces)
 - ...

GNU Core Utilities

head	cat	sum	column	sort	join	expand
tail	tac	cksum	paste	uniq	comm	unexpand
	cut	md5sum			tr	split

Why do we need them?

- In science, most of our time in the terminal is spent acting on files
- Mastering file handling in general can allow us to simplify the data analysis software
- Typical operations might be
 - Prepare data for plotting in a given format
 - Search patterns in a file
 - Extract portion of a file
 - Transform a file (e.g. remove trailing spaces)
 - ...



Infinite Power!

GNU Core Utilities

head cat column sort expand
tail paste unexpand
sed awk
split

Sed: A marvellous utility

The awful truth about `sed`

Sed is extremely powerful. Unfortunately, most people never learn its real power. The language is simpler than the average user thinks, because the documentation is far from being ideal. Sed has several commands, but most people only learn the substitute command 's'. And the GNU manual begins exactly with examples about the 's' command.

Not bad!

The substitute command is just one command. Sed has at least 20 different commands for you. You can even write  Tetris in it (not to mention that it's Turing complete).

Some references

 The official GNU manual { The PDF of v4.10 is 88 pages long... }

 A very nice tutorial { Bash code there often uses deprecated features, but you should know it by now! }

 One-liners, with some comments

 More one-liners, explained but advanced!

The PDF manual is 88 pages long. What should I learn here?

- We will focus on the abstract idea of **how** `sed` processes a file
- Some of the simplest commands (acting on a single line) will be introduced
- Learning Sed is almost like learning a new programming language, mastering it is probably not needed for a physicist either

The PDF manual is 88 pages long. What should I learn here?

- We will focus on the abstract idea of **how** `sed` processes a file
- Some of the simplest commands (acting on a single line) will be introduced
- Learning Sed is almost like learning a new programming language, mastering it is probably not needed for a physicist either

```
# Invocation
sed [OPTIONS...] [SCRIPT] [INPUTFILE...]

# General command syntax to be put in [SCRIPT]
[addr]X[options]
```

The PDF manual is 88 pages long. What should I learn here?

- We will focus on the abstract idea of **how** `sed` processes a file
- Some of the simplest commands (acting on a single line) will be introduced
- Learning Sed is almost like learning a new programming language, mastering it is probably not needed for a physicist either

```
# Invocation
sed [OPTIONS...] [SCRIPT] [INPUTFILE...]

# General command syntax to be put in [SCRIPT]
[addr]X[options]
```

- X is a `sed` command
- [addr] is an optional line address
If [addr] is specified, the command X will be executed only on the matched lines;
[addr] can be a single line number, a regular expression, or a range of lines
- Additional [options] are used for some `sed` commands
- Quote the commands to avoid shell globbing conflicts
→ usually you want to use single quotes

Some of the most used commands

`s/regexp/replacement/[flags]`

Match the regular-expression against the content of the pattern space.
If found, replace matched string with replacement.

`p`

Print the pattern space, up to the first newline.

`d`

Delete the pattern space; immediately start next cycle.

`=`

Print the current input line number (with a trailing newline).

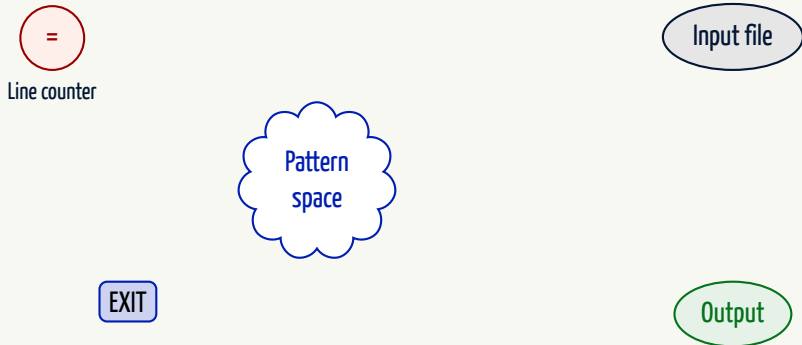
`{ cmd ; cmd ... }`

Group several commands together.

`q`

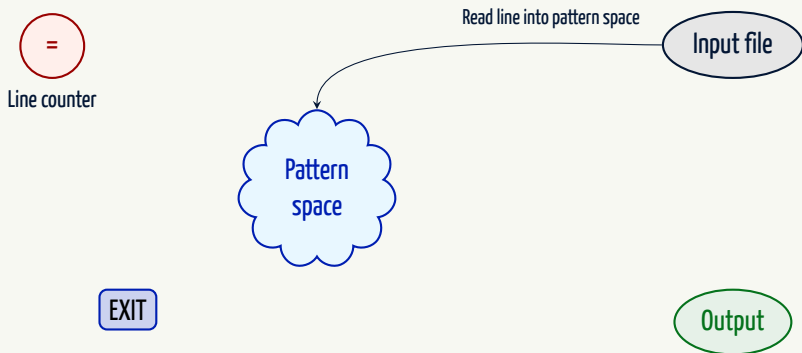
Exit `sed` without processing any more commands or input.

A sed cycle: Being line oriented



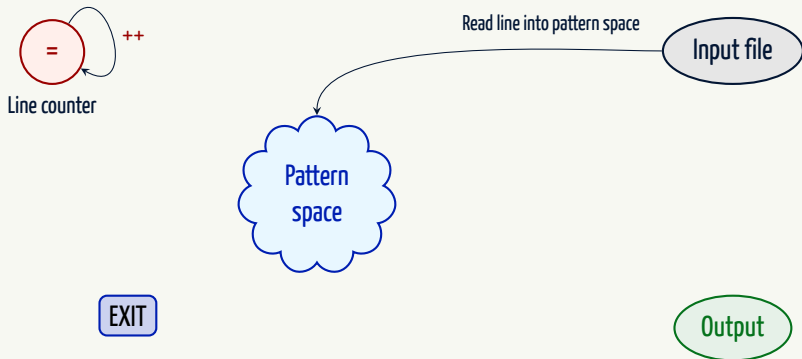
A sed cycle: Being line oriented

- 1 The next line is read from the input file and placed it in the pattern space.
If the end of file is found, and if there are additional files to read, the current file is closed, the next file is opened, and the first line of the new file is placed into the pattern space.



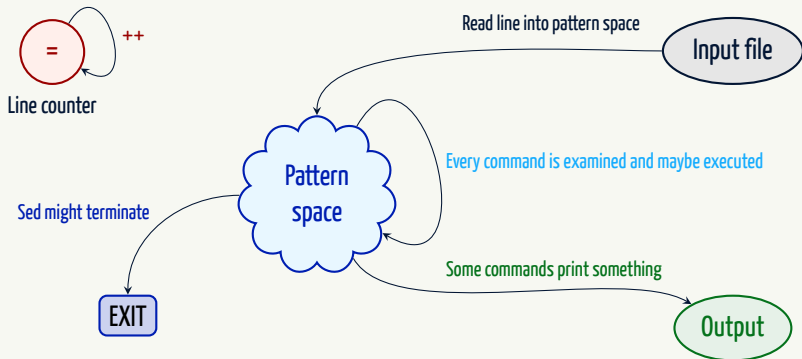
A sed cycle: Being line oriented

- 2 The line count is incremented by one.
Opening a new file does not reset this number.



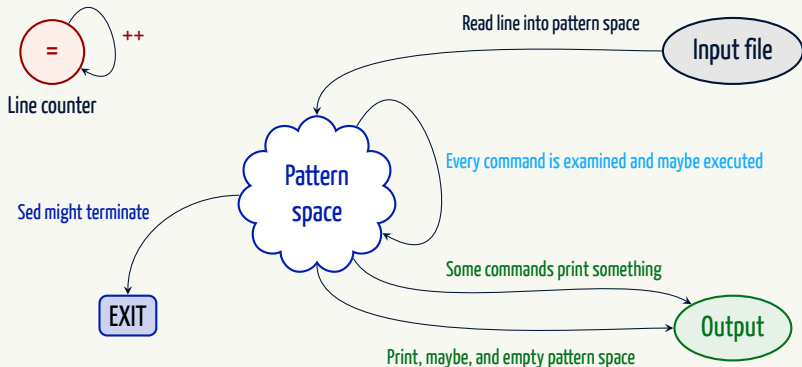
A sed cycle: Being line oriented

- 3 Each `sed` command is examined. If there is a restriction placed on the command, and the current line in the pattern space meets that restriction, the command is executed. Some commands, like 'd' cause `sed` to go to the top of the loop. The 'q' command causes `sed` to stop. Otherwise the next command is examined.



A sed cycle: Being line oriented

- 4 After all of the commands are examined, the pattern space is printed to the output (unless sed has the optional `-n` argument) and the pattern space is emptied.



Address restrictions

A command can be limited to some lines using restrictions

- A positive line number (use \$ to refer to the last line)
- A regular expression `/regex/` or `\•regex•` where `•` is any character { not to be matched! }
- Use an exclamation mark after the address to negate that address
- A comma separated pair to specify a range of lines

```
3          # Command applies
/^#/       # - to line 3 only
\%^//%     # - to lines starting by #
5!         # - to lines starting by //
1,3        # - not to line 5
3,$        # - to first three lines
1,/one/    # - from line 3 to the end
/begin/,/end/ # - till the first line matching 'one'
           # - from the first line matching 'begin'
           # - to the first matching 'end' (included)
```

The main commands: Substitute

`s/regexp/replacement/[flags]`

The 's' command attempts to match the pattern space against the supplied regular expression `regexp`; if the match is successful, then that portion of the pattern space which was matched is replaced with `replacement`.

- The `/` is called delimiter and can be replaced with any character
- The delimiter can appear in `regexp` or `replacement` only if escaped
- Unescaped `&` characters reference the whole matched portion of the pattern space
- The replacement can contain `\1, ..., \9` references, which refer to the portion of the match which is contained between the first, ..., ninth `\(` and its matching `\)`
- Important-to-know flags are
 - `g` Apply the replacement to all matches to the `regexp`, not just the first
 - `number` Only replace the number-th match of the `regexp` (between 1 and 512)
 - `p` If the substitution was made, then print the new pattern space

The main commands: Substitute

```
1 $ printf '11\n12\n' | sed 's/12/twelve/'
2 11
3 twelve
4 $ printf '11\n12\n' | sed 's/1/X/'
5 X1
6 X2
7 $ printf '11\n12\n' | sed 's/1/X/g'
8 XX
9 X2
10 $ printf '11\n12\n' | sed 's/1/X/2'
11 1X
12 12
13 $ printf '11\n12\n' | sed 's/2/Y/p'
14 11
15 1Y
16 1Y
17 $ sed 's/[0-9]\+/(&)/' <<< 'Wed 09 Sep 2019'
18 Wed (09) Sep 2019
19 $ sed 's/[0-9]\+/(&)/g' <<< 'Wed 09 Sep 2019'
20 Wed (09) Sep (2019)
21 $ sed -r 's/[0-9]\+/(&)/g' <<< 'Wed 09 Sep 2019'
22 Wed (09) Sep (2019) # -r option for ERE
```

The main commands: Substitute

```
23 $ printf '11\n12\n' | sed 's/1/&&/'
24 111
25 112
26 $ sed 's/\([a-z]\+\) \([a-z]\+\)/\2 \1/' <<< 'hello world'
27 world hello
28 $ sed 's/^\(.\)\(.\)\(.\)\(.\) /\3\2\1\4/' <<< 'nice star'
29 cinestar
30 $ printf "=%.0s" {1..20} | sed 's/./&:/10'; echo
31 =====:=====
32 $ cd /usr/local/bin
33 $ sed 's/\usr\//local\//bin\//common\//bin/' <<< "${PWD}/emacs"
34 /common/bin/emacs
35 $ sed 's_/usr/local/bin_/common/bin_' <<< "${PWD}/emacs"
36 /common/bin/emacs
37 $ sed 's:/usr/local/bin:/common/bin:' <<< "${PWD}/emacs"
38 /common/bin/emacs
39 $ sed 's|/usr/local/bin|/common/bin|' <<< "${PWD}/emacs"
40 /common/bin/emacs
41 $ value='3 5'; sed 's/Y/"${value}"/' <<< "XYZ"
42 X3 5Z
43 $ value='3 5'; sed 's/Y/"${value}"/' <<< "XYZ"
44 sed: -e expression #1, char 5: unterminated `s' command
```

The main commands: Print

- Print out the pattern space
- Note that the printing induced by the `p` command is unrelated to the printing done by `sed` at the end of the cycle
- Hence, this command is usually only used in conjunction with the `-n` command-line option of `sed` (whose long name is `--quiet` or `--silent`)
- It is commonly used together with an address restriction

```
1 $ sed 'p' <<< 'Echo me'
2 Echo me
3 Echo me
4 $ sed -n 'p' <<< 'Only once'
5 Only once
6 $ printf '%d\n' {1..9} | sed -n '5 p'
7 5
8 $ printf '%d\n' {1..9} | sed -n '7,$ p'
9 7
10 8
11 9      # Try out: printf '%d\n' {1..10} | sed -n '1~3 p'
```

The main commands: Delete

- Delete the pattern space and **immediately start next cycle**
- It is used to delete lines from the input file(s)
- It is commonly used together with an address restriction

```
1 $ printf '%d\n' {1..10} | sed '3,$ d'
2 1
3 2
4 $ sed '2 d' <<< $'A\n nice but\n cold place'
5 A
6 cold place
7 $ sed '/b/,/f/ d' <<< $'a\nb\nc\nd\ne\nf\ng'
8 a
9 g
10 $ sed '/^#/ d' filename # Remove Bash comments
11 # ... <- look up option -i of sed to edit the file in place
12 $ sed '/^$/ d' filename # Remove empty lines
13 # Print only lines with less than 65 characters
14 $ sed '/^.{65\}/d' filename
```


The main commands: quit, line number and groups

- The 'q' command exits `sed` without processing any more commands or input. This command exits at the end of the cycle, i.e. it prints the current pattern space.
- The '=' command prints out the current input line number (with a trailing newline).
- A group of commands may be enclosed between { and } characters. This is particularly useful when you want a group of commands to be triggered by a single address (or address-range) match.

```
1 $ printf '%d\n' {1..15} | sed '2q'
2 1
3 2
4 $ sed '=' <<< '$aaa\nbbb'
5 1
6 aaa
7 2
8 bbb
9 $ sed -n '$=' filename # Equivalent to 'wc -l < filename'
10 $ printf '%d\n' {1..3} | sed -n '2{s/2/X/ ; p}'
11 X
```

Exercise sheet 4 (II)

Some guidance about the session

- You have roughly 60 minutes
 - ↳ You should manage to do the new exercises
 - ↳ If you are done earlier, do the rest!
- Each exercise should be solved in a dedicated script { Use the terminal to experiment, if needed }
- **Now you can tackle all exercises**
- You can also finish up the work from the last practice session
 - ↳ Consider this as further practice on this topic
- Note down your questions, doubts, curiosities, ...
- **If you can't resist: Give your solution to an AI to get feedback** { The trainer can also give you feedback }

Come up with suggestions!

In the closing sessions we'll solve an exercise together.
If you have any preference, feel free to speak up.